

SUMMARY

This topic covers a technical briefing on the fundamentals of Tree Data Structures, including terminology, structural properties, binary tree constraints, implementation, and traversal methodologies. Trees are essential for representing hierarchical relationships in computing and are widely applied in file systems, databases, compilers, and search optimization.

Trees are non-linear data structures that organize data hierarchically. Key takeaways include:

- Trees model parent-child relationships and are widely used in file systems, organizational charts, HTML/DOM structures, and database indexing.
- Binary trees restrict each node to at most two children, allowing mathematical analysis of height and node density.
- Tree nodes are commonly implemented using a linked representation with pointers to children and a root pointer to access the structure.
- Traversal techniques—Pre-order, In-order, Post-order, and Level-order—leverage recursion to process all nodes efficiently.

1. INTRODUCTION TO TREE DATA STRUCTURES

A tree is a non-linear data structure with hierarchical organization.

1.1 Practical Applications

- System Structures: File and folder organization.
- Organizational Charts: Management hierarchies.
- Web Technologies: HTML, XML, JSON (DOM).
- Database Management: B-Trees and B+ Trees for indexing.
- Compiler Design: Parse trees.
- Search Optimization: Binary Search Trees (BSTs).

2. CORE STRUCTURAL TERMINOLOGY

Term	Definition
Node	Element in a tree, often represented as a circle containing data.
Child	Nodes directly connected below a parent node.
Parent	Node directly above a child node.
Root	Top-most node with no parent.
Leaf Node	Node with zero children.

Internal Node Node with at least one child.

Sibling Nodes sharing the same parent.

Ancestor Parent of a node, parent of parent, up to root.

Descendant Children of a node and all subsequent generations.

2.1 The Concept of Degree

- Degree of a node = number of children.
- Leaf node degree = 0.
- Node with n children = degree n.

3. SPATIAL MEASUREMENTS: HEIGHT AND DEPTH

3.1 Height

- Height of node x = length of longest path from x to a leaf.
- Leaf node height = 0.
- Height of tree = height of root.
- Formula: $\text{Height}(\text{node}) = 1 + \max(\text{height of children})$.

3.2 Depth

- Depth of node x = length of path from root to x.
- Root depth = 0.
- Formula: $\text{Depth}(\text{child}) = \text{Depth}(\text{parent}) + 1$.

4. BINARY TREES

Binary trees restrict each node to at most two children.

4.1 Node Density and Height Constraints

- Minimum nodes for height h: $h + 1$.
- Maximum nodes for height h: $2^{\{h+1\}} - 1$ (Perfect Binary Tree).

4.2 Structural Varieties

- Perfect Binary Tree: All internal nodes have two children; all leaves at same level.
- Complete k-ary Tree: Every internal node has k children. Total nodes: $N = k * I + 1$ (I = internal nodes).
- Leaf nodes in binary trees: $n_0 = n_2 + 1$ (nodes with two children + 1).

5. IMPLEMENTATION AND REPRESENTATION

- **Linked Representation:** Each node contains:
 1. Data (value).
 2. Left pointer (address of left child or NULL).
 3. Right pointer (address of right child or NULL).
- Root pointer provides entry to all tree operations.

6. TREE TRAVERSAL METHODOLOGIES

6.1 Level Order Traversal

- Visits nodes level-by-level (Breadth-First Search).

6.2 Depth-Based Traversal (Recursive)

Traversal Type	Order of Visitation	Description
Pre-order	Root → Left → Right	Process root before subtrees.
In-order	Left → Root → Right	Process root between left and right subtrees.
Post-order	Left → Right → Root	Process root after all subtrees.

- Traversals leverage recursion: any node can itself be treated as a root of its subtree.